

09

SEÇÃO 9



TRATAMENTO DE DADOS

100 exercícios em sequência | progressivos | sem gabarito

SEU DESAFIO

Leia cada solicitação, localize os dados necessários e construa a consulta. Os enunciados treinam toda a seção e combinam conteúdos já estudados.

CONVERSÃO

CASE

COALESCE

TEXTOS

DATAS

FUNÇÕES

01 Converta o texto 2026-09-15 para DATE e retorne o resultado como data_convertida.

02 Converta o texto 150000.50 para NUMERIC e retorne o resultado como valor_convertido.

03 Converta o texto 2024 para INTEGER, some 1 e retorne ano_seguinte.

04 Retorne product_id, brand, model, price e CAST(price AS VARCHAR) como price_texto.

05 Retorne city, state, population e CAST(population AS VARCHAR) como population_texto.

06 Retorne visit_id, visit_page_date e CAST(visit_page_date AS TIMESTAMP) como acesso_timestamp.

07 Converta DATE 2026-01-31 para TIMESTAMP e retorne data_original e data_hora.

08 Retorne product_id, model_year e CAST(model_year AS INTEGER) como ano_numerico.

09 Calcule a proporção de clientes clt sobre o total com conversão decimal e retorne proporcao_clt.

10 Calcule a proporção de visitas pagas sobre o total evitando divisão inteira e retorne proporcao_paga.

- 11 Calcule a proporção de clientes de SP sobre o total com casas decimais e retorne `proporcao_sp`.
- 12 Use o operador de conversão abreviado para transformar 9876.54 em NUMERIC e retorne `valor_numerico`.
- 13 Retorne `product_id`, `price` e `CAST(price AS INTEGER)` como `price_inteiro`.
- 14 Retorne `customer_id`, `birth_date` e `birth_date` convertido para TIMESTAMP como `nascimento_timestamp`.
- 15 Retorne `product_id` e a descrição `brand || espaço || model || espaço || CAST(model_year AS VARCHAR)` como `descricao_com_ano`.
- 16 Classifique `income` em `abaixo_5000`, `5000_a_9999`, `10000_a_14999`, `15000_a_19999` ou `20000_ou_mais`; retorne `customer_id`, `income` e `faixa_renda`.
- 17 Classifique `score` em `baixo` para valores abaixo de 400, `medio` de 400 a 699 e `alto` a partir de 700; retorne `customer_id`, `score` e `faixa_score`.
- 18 Classifique `price` em `popular` abaixo de 60000, `intermediario` de 60000 a 119999 e `alto` a partir de 120000; retorne `product_id`, `price` e `faixa_preco`.
- 19 Classifique `population` preenchida em `pequena` abaixo de 50000, `media` de 50000 a 499999 e `grande` a partir de 500000; retorne `city`, `state`, `population` e `faixa_populacao`.
- 20 Traduza `professional_status` para português nos valores `clt`, `freelancer`, `retired`, `businessman`, `self_employed`, `civil_servant` e `student`; use outro nos demais e retorne `customer_id`, `professional_status` e `status_pt`.

21 Classifique cada visita como paga quando paid_date estiver preenchida e nao_paga caso contrário; retorne visit_id, paid_date e situacao_pagamento.

22 Classifique a etapa mais avançada da visita como pagamento, checkout, carrinho ou acesso; retorne visit_id e etapa_maxima.

23 Classifique discount em sem_desconto para zero ou positivo, ate_10 para -0.10 a abaixo de zero, de_10_a_20 para -0.20 a abaixo de -0.10 e acima_20 para valores menores que -0.20; retorne visit_id, discount e faixa_desconto.

24 Converta model_year para INTEGER e classifique veículos como antigo antes de 2020 ou recente a partir de 2020; retorne product_id, model_year e classificacao_ano.

25 Classifique clientes como Sudeste para SP, RJ, MG ou ES e Outras_regioes nos demais states; retorne customer_id, state e macro_regiao.

26 Classifique city como metropole_calculada quando population superar 1000000; nos demais casos use region; retorne city, state, population e classificacao.

27 Classifique clientes como prioridade_alta se income superar 15000 ou score superar 800, prioridade_media se income superar 8000 ou score superar 600, e prioridade_padrao nos demais; retorne customer_id, income, score e prioridade.

28 Converta model_year para INTEGER e classifique-o como ano_par ou ano_impar usando resto da divisão por 2; retorne product_id, model_year e paridade_ano.

29 Preserve todas as lojas e classifique-as como com_pagamento ou sem_pagamento pela contagem de paid_date; retorne store_id, store_name e situacao_venda.

30 Preserve todos os clientes e indique correspondencia_regional como sim ou nao após comparar state e city sem diferença de caixa; retorne customer_id, city, state e correspondencia_regional.

31 Classifique cada visita como abandonou_no_acesso, abandonou_no_carrinho, abandonou_no_checkout ou comprou; retorne visit_id e desfecho_funil.

32 Aplique desconto de 10% ao price e classifique o resultado em ate_50000, 50000_a_100000 ou acima_100000; retorne product_id, preco_promocional e faixa_promocional.

33 Classifique discount como positivo_ou_zero, negativo_ate_10, negativo_10_a_20 ou negativo_acima_20; retorne visit_id, discount e risco_desconto.

34 Classifique clientes como premium se income maior que 15000 e score maior que 700, potencial se income maior que 9000 ou score maior que 600, e padrao nos demais; retorne customer_id, income, score e segmento.

35 Atribua ordem_etapa 4 para pagamento, 3 para checkout, 2 para carrinho e 1 para acesso; retorne visit_id e ordem_etapa.

36 Traduza gender female para feminino e male para masculino; retorne first_name, gender e genero_pt de temp_tables.ibge_genders.

37 Converta model_year para INTEGER e classifique-o em ate_2010, 2011_a_2015, 2016_a_2020 ou apos_2020; retorne product_id, model_year e faixa_ano.

38 Para visitas pagas, calcule paid_date - visit_page_date e classifique em mesmo_dia, ate_7_dias ou mais_de_7_dias; retorne visit_id, dias_ate_pagamento e faixa_tempo.

39 Classifique population como sem_informacao quando nula, ate_10000, 10001_a_100000 ou acima_100000; retorne city, state, population e faixa_populacao.

40 Em temp_tables.duplicados, conte linhas por cpf e classifique como unico ou repetido; retorne cpf, total_linhas e situacao_cpf.

41 Retorne city, state, population e COALESCE(population,0) como population_tratada.

42 Substitua population nula pela média das populações preenchidas e retorne city, state, population e population_tratada.

43 Retorne visit_id, add_to_cart_date e COALESCE(add_to_cart_date,visit_page_date) como data_carrinho_tratada.

44 Retorne visit_id, start_checkout_date e COALESCE(start_checkout_date,add_to_cart_date,visit_page_date) como data_checkout_tratada.

45 Retorne visit_id e a primeira data disponível entre paid_date, finish_checkout_date, start_checkout_date, add_to_cart_date e visit_page_date como data_mais_avancada.

46 Retorne visit_id, discount e COALESCE(discount,0) como discount_tratado.

47 Retorne customer_id, income e COALESCE(income,0) como income_tratada sem alterar a tabela.

48 Retorne customer_id, score e COALESCE(score,0) como score_tratado.

49 Retorne store_id e COALESCE(store_name,'Loja sem nome') como store_name_tratado.

50 Preserve todas as visitas ao relacioná-las a clientes e retorne visit_id e COALESCE(first_name,'Cliente desconhecido') como nome_cliente.

51 Preserve todas as visitas ao relacioná-las a veículos e retorne visit_id e COALESCE(model,'Produto não cadastrado') como modelo_veiculo.

52 Preserve todos os clientes ao relacioná-los a regiões e retorne customer_id, city, state e COALESCE(region,'Região não identificada') como region_tratada.

53 Relacione visitas e veículos e calcule price * (1 + COALESCE(discount,0)) como preco_com_desconto; retorne visit_id, price, discount e preco_com_desconto.

54 Calcule taxa_pagamento por loja com divisão decimal; retorne store_id, total_visitas, visitas_pagas e taxa_pagamento.

55 Preserve todos os clientes ao relacioná-los a gêneros e retorne customer_id, first_name e COALESCE(gender,'unknown') como gender_tratado.

56 Retorne customer_id, first_name e LOWER(first_name) como first_name_minusculo.

57 Retorne customer_id, last_name e UPPER(last_name) como last_name_maiusculo.

58 Retorne store_id, store_name e TRIM(store_name) como store_name_limpo.

59 Retorne customer_id, city e REPLACE(city,'SÃO PAULO','São Paulo') como city_exibicao.

60 Retorne customer_id, professional_status e LOWER(TRIM(professional_status)) como status_padronizado.

61 Retorne `customer_id`, `city` e `UPPER(TRIM(city))` como `city_padronizada`.

62 Retorne `store_id`, `store_name` e `REPLACE(TRIM(store_name), ' ', '-')` como `store_name_com_hifens`.

63 Filtre a cidade de São Paulo sem depender de caixa e retorne `customer_id`, `first_name`, `city` e `state`.

64 Retorne `customer_id` e `UPPER(TRIM(first_name)||' '||TRIM(last_name))` como `nome_completo_maiusculo`.

65 Retorne `product_id` e `UPPER(TRIM(brand)||'-'||TRIM(model))` como `veiculo_padronizado`.

66 Retorne `customer_id`, `nome_completo` e `LENGTH(nome_completo)` como `total_caracteres`, usando `nome` e `sobrenome` limpos.

67 Retorne `store_id`, `store_name` e `LENGTH(TRIM(store_name))` como `total_caracteres`.

68 Retorne `product_id`, `brand` e `LEFT(brand,3)` como `sigla_brand`.

69 Retorne `visit_id` e `RIGHT(visit_id,4)` como `identificador_curto`.

70 Retorne `store_id` e `LOWER(REPLACE(store_name, ' ', '-'))` como `slug_loja`.

71 Retorne `customer_id` e `UPPER(TRIM(first_name)||' - '||TRIM(city)||'/'||state)` como `rotulo_cliente`.

72 Relacione clientes e regiões comparando `UPPER(TRIM(city))` e `state`; retorne `customer_id`, `city_cliente`, `city_regiao`, `state` e `region`.

73 Agrupe clientes por `LOWER(TRIM(professional_status))` e retorne `status_padronizado` e `total_clientes`.

74 Filtre `store_name` contendo `AUTO` após `UPPER` e retorne `store_id`, `store_name` e `UPPER(store_name)` como `store_name_maiusculo`.

75 Retorne `product_id`, `brand`, `model` e `REPLACE(brand||' '||model,' ','-')` como `descricao_com_hifens`.

76 Retorne `visit_id`, `visit_page_date` e `visit_page_date + INTERVAL '7 days'` como `acompanhamento_7_dias`.

77 Retorne `visit_id`, `visit_page_date` e `visit_page_date - INTERVAL '30 days'` como `data_30_dias_antes`.

78 Retorne `visit_id`, `visit_page_date` e `DATE_TRUNC('month',visit_page_date)` como `inicio_mes`.

79 Retorne `visit_id`, `visit_page_date` e `DATE_TRUNC('week',visit_page_date)` como `inicio_semana`.

80 Retorne `visit_id`, `visit_page_date` e `DATE_TRUNC('year',visit_page_date)` como `inicio_ano`.

81 Retorne `visit_id`, `visit_page_date` e `EXTRACT(YEAR FROM visit_page_date)` como `ano_acesso`.

82 Retorne `visit_id`, `visit_page_date` e `EXTRACT(MONTH FROM visit_page_date)` como `mes_acesso`.

83 Retorne `visit_id`, `visit_page_date` e `EXTRACT(DAY FROM visit_page_date)` como `dia_acesso`.

84 Retorne `visit_id`, `visit_page_date` e `EXTRACT(DOW FROM visit_page_date)` como `dia_semana_numero`.

85 Conte visitas por ano e mês de `visit_page_date`; retorne `ano_acesso`, `mes_acesso` e `total_visitas`.

86 Conte visitas por `DATE_TRUNC('week', visit_page_date)`; retorne `inicio_semana` e `total_visitas` em ordem cronológica.

87 Conte pagamentos por `DATE_TRUNC('month', paid_date)`; retorne `inicio_mes` e `total_pagamentos`, ignorando `paid_date` nula.

88 Conte visitas por `EXTRACT(DOW FROM visit_page_date)`; retorne `dia_semana_numero` e `total_visitas` ordenados pelo número.

89 Para visitas pagas, retorne `visit_id`, `visit_page_date`, `paid_date` e `paid_date - visit_page_date` como `dias_ate_pagamento`.

90 Para visitas com checkout, retorne `visit_id`, `add_to_cart_date`, `start_checkout_date` e a diferença em dias como `dias_carrinho_checkout`.

91 Converta `visit_page_date` e `paid_date` para `DATE` e calcule a diferença; retorne `visit_id`, `acesso_data`, `pagamento_data` e `dias_ate_pagamento`.

92 Calcule a data de acompanhamento sete dias após carrinho ou, se nulo, após acesso; retorne `visit_id` e `data_acompanhamento`.

93 Liste visitas cujo mês de `paid_date` seja diferente do mês de `visit_page_date`; retorne `visit_id`, `visit_page_date` e `paid_date`.

94 Liste visitas cujo `start_checkout_date` ocorreu no mesmo dia de `add_to_cart_date`; retorne `visit_id`, `add_to_cart_date` e `start_checkout_date`.

95 Para visitas pagas, agrupe por `dias_ate_pagamento` e retorne `dias_ate_pagamento` e `total_visitas`.

96 Relacione clientes e regiões com `UPPER(TRIM(city))`; retorne `customer_id`, `nome_completo` em maiúsculas, `city_regiao`, `state`, `region` e `population_tratada` com zero nos nulos.

97 Converta `model_year` para `INTEGER`, calcule o resto por 2 e classifique a paridade; retorne `product_id`, `ano_numerico`, `resto_2` e `paridade`.

98 Crie um contato padronizado e retorne `customer_id`, `nome_completo`, `email_minusculo` e `mobile_tratado` com Sem telefone quando nulo.

99 Por mês de acesso, retorne `inicio_mes`, `total_visitas`, `visitas_pagas` e `taxa_pagamento` com divisão decimal.

100 Retorne `visit_id`, `data_referencia` pela ordem pagamento, checkout, carrinho e acesso, e uma `situacao` final paga ou não paga.